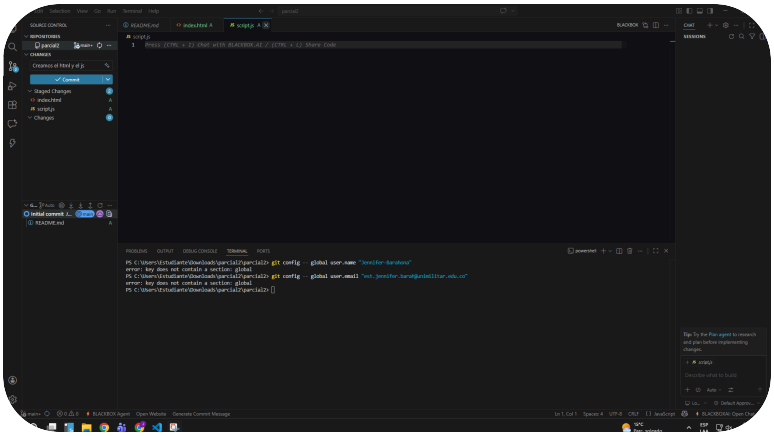

no me deja poner el ususario y el gmail como le hago?

El problema es simplemente un error de sintaxis en el comando 👀

Estás escribiendo:

```
git config -- global user.name "Jennifer-Barahona"
```

Pero ese espacio entre `--` y `global` no debe ir.

✅ Comando correcto:

```
git config --global user.name "Jennifer-Barahona"
git config --global user.email "est.jennifer.barah@unimilitar.edu.co"
```

🔴 ¿Por qué te da error?

Git interpreta `--` como una opción incompleta, y por eso dice:

```
error: key does not contain a section: global
```

💡 Verifica que sí quedó guardado:

Después de ponerlos bien, ejecuta:

```
git config --global --list
```

Deberías ver algo como:

```
user.name=Jennifer-Barahona
user.email=est.jennifer.barah@unimilitar.edu.co
```

Si quieres, luego te ayudo a conectar tu repositorio con GitHub o hacer tu primer push 🚀

cual es el comando para crear la estructura basica del html

No hay un **comando universal de Git o la terminal** para crear la estructura básica de HTML, pero sí tienes varias formas fáciles 👇

✅ 1. La forma más rápida (en Visual Studio Code)

Solo escribe en tu archivo `.html`:

```
!
```

y luego presiona:

```
Tab
```

✨ Eso usa **Emmet** y te genera automáticamente algo como:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
```

HTML

✅ 2. Crear el archivo desde la terminal

Si estás en la carpeta del proyecto:

Bash

(o en Windows PowerShell):

PowerShell

Luego lo abres en VS Code y usas `! + Tab`.

✅ 3. Escribirlo manualmente

Si no quieres usar atajos:

```
<!DOCTYPE html>
<html>
<head>
    <title>Mi página</title>
</head>
<body>
```

HTML

🔥 Recomendación

Usa siempre `! + Tab` en Visual Studio Code porque:

- Es más rápido
- Evitas errores
- Ya incluye configuraciones modernas

Si quieres, te enseño cómo conectar ese HTML con tu `script.js` o tu `canvas` (vi que estás trabajando con eso 😉).

puedes darme una funcion para crear numeros random entre 5 y 10 porfa

Claro 👍 aquí tienes una función en **JavaScript** para generar números aleatorios entre 5 y 10:

🎯 Números enteros (los más usados)

```
function randomEntre5y10() {
```

JavaScript

🔍 Cómo funciona:

- `Math.random()` → da un número entre 0 y 1
- `* 6` → convierte el rango a 0–5
- `+ 5` → desplaza el rango a 5–10

✔️ Resultado: números enteros entre **5 y 10 (incluidos)**

🎯 Si quieres decimales

```
function randomDecimalEntre5y10() {
```

JavaScript

✔️ Resultado: número entre **5 y 10**, pero con decimales

🧪 Ejemplo de uso:

JavaScript

Si quieres, te puedo hacer una función más general tipo:
👉 `random(min, max)` para cualquier rango (te sirve más a largo plazo).

como es el algoritmo de punto medio?

El **algoritmo de punto medio** (Midpoint) se usa mucho en computación gráfica para dibujar figuras (líneas, círculos) de forma eficiente usando solo enteros.

Te explico el más común: **para dibujar círculos** 👇

🟢 Idea principal

En vez de calcular puntos con ecuaciones complicadas, el algoritmo:

- Va punto por punto
- Decide si el siguiente píxel va **arriba o abajo**
- Usa una variable de decisión llamada **punto medio**

🔵 Ecuación base del círculo

Un círculo centrado en (0,0) cumple:

$$x^2 + y^2 = r^2$$

El algoritmo evalúa un punto intermedio para decidir hacia dónde avanzar.

🧠 Lógica

Empiezas en:

```
x = 0
y = r
```

Y vas avanzando en `x` mientras decides si bajar `y`.

⚙️ Algoritmo (paso a paso)

```javascript
function dibujarCirculo(cx, cy, r, ctx) {
    let x = 0;
    let y = r;
    let d = 1 - r;

    while (x <= y) {
        // Dibujar los 8 puntos simétricos
        dibujarPuntos(cx, cy, x, y, ctx);

        if (d < 0) {
            d = d + (2 * x) + 3;
        } else {
            d = d + (2 * (x - y)) + 5;
            y--;
        }
        x++;
```

JavaScript

✳️ Función para simetría (MUY importante)

```javascript
function dibujarPuntos(cx, cy, x, y, ctx) {
    ctx.fillRect(cx + x, cy + y, 1, 1);
    ctx.fillRect(cx - x, cy + y, 1, 1);
    ctx.fillRect(cx + x, cy - y, 1, 1);
    ctx.fillRect(cx - x, cy - y, 1, 1);
    ctx.fillRect(cx + y, cy + x, 1, 1);
    ctx.fillRect(cx - y, cy + x, 1, 1);
    ctx.fillRect(cx + y, cy - x, 1, 1);
```

JavaScript

💡 ¿Por qué 8 puntos?

Porque el círculo es simétrico en 8 direcciones, entonces:

👉 calculas 1 punto

👉 dibujas 8

🚀 Mucho más eficiente

🔴 ¿Qué hace la variable `d`?

Es el "decisor":

- Si `d < 0` → eliges el punto de arriba
- Si `d >= 0` → bajas en `y`

🟣 Resumen corto

- Empiezas en (0, r)
- Avanzas en x
- Decides si bajar y usando `d`
- Dibujas 8 puntos simétricos

Si quieres, te explico también:

✅ ↳ el de **líneas (Bresenham)**

✅ o ↳ cómo integrarlo con tu canvas (vi que estás usando `ctx`)

como es el comando para organizar el codigo

Si estás usando Visual Studio Code, el comando para **organizar / formatear el código automáticamente** es:

✅ Atajo de teclado

💻 Windows / Linux:

Bash

🍎 Mac:

Bash

✅ Otra forma (con clic derecho)

1. Clic derecho dentro del código
2. Selecciona "**Format Document**" (Formatear documento)

⚠️ Si no te funciona

Probablemente te falta un formateador. Puedes instalar uno como:

👉 Prettier

Luego:

1. Instálalo en extensiones
2. Vuelve a usar `Shift + Alt + F`

💡 Tip extra (muy útil)

Puedes hacer que se ordene automáticamente al guardar:

1. Ve a configuración (`Ctrl + ,`)
2. Busca: `format on save`
3. Activa ✔ **Format On Save**

Si quieres, también ↪ te puedo ayudar a que tu código de `canvas` se vea más limpio (indentación, nombres, etc.).

```
/**
 * Universidad UMNG - Facultad de Ingeniería Multimedia
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: Jennifer Barahona Suarez
 * * Tarea: Implementar los algoritmos de rasterización
manual.
 */

//Obtenemos el contexto del canvas para dibujar
const canvas = document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");

//Funcion para crear un número aleatorio entre 5 y 10
function randomEntre5y10() {
    return Math.floor(Math.random() * 6) + 5;
}
```

```
// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#a200ff") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */

function bresenhamLine(x0, y0, x1, y1, color) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx;
    if (x0 < x1) {
        sx = 1;
    } else {
        sx = -1;
    }

    let sy;
    if (y0 < y1) {
        sy = 1;
    } else {
        sy = -1;
    }

    let error = dx - dy; // parámetro de decisión inicial

    while (true) {
        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;
```

```
        let e2 = 2 * error;

        /**
         * Ajuste del error:
         * Si el error acumulado supera cierto umbral,
         * se corrige desplazando en X o Y para aproximarse
         * a la recta ideal.
         */
        if (e2 > -dy) {
            error -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            error += dx;
            y0 += sy;
        }
    }
}

/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */

function getPolygonVertices(centerX, centerY, sides, radius)
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
    let vertices = [];

    for (let i = 0; i < sides; i++) {
        let angulo = (2 * Math.PI * i) / sides;
        let x = centerX + radius * Math.cos(angulo); // Cálculo
de la coordenada X usando coseno
        let y = centerY + radius * Math.sin(angulo); // Cálculo
de la coordenada Y usando seno
```

```
        vertices.push({ x: Math.round(x), y: Math.round(y) });
    }

    return vertices;// Retorna un arreglo con las
coordenadas de los vértices
}



/**
 * Algoritmo del punto medio para circunferencias
 * cx, cy: centro de la circunferencia
 * r: radio de la circunferencia
 * color: color de la circunferencia
 * El parámetro p decide cuándo decrementar Y
 */
function drawCircle(cx, cy, r, color = "#13bddb") {
    let x = 0;
    let y = r;
    let p = 1 - r;

    while (x <= y) {
        plotCirclePoints(cx, cy, x, y, color);

        x++;

        if (p < 0) {
            p += 2 * x + 1;
        } else {
            y--;
            p += 2 * (x - y) + 1;
        }
    }
}

function plotCirclePoints(cx, cy, x, y, color) {
    drawPixel(ctx, cx + x, cy + y, color);
    drawPixel(ctx, cx - x, cy + y, color);
    drawPixel(ctx, cx + x, cy - y, color);
    drawPixel(ctx, cx - x, cy - y, color);
```

```
    drawPixel(ctx, cx + y, cy + x, color);
    drawPixel(ctx, cx - y, cy + x, color);
    drawPixel(ctx, cx + y, cy - x, color);
    drawPixel(ctx, cx - y, cy - x, color);
}
```

que le corregirias a este codigo? no me des el codigo si no dime solo puntualmente que cambiarias o que puede faltar

Tu código está **bien estructurado y correcto en lo esencial**, pero hay varios detalles puntuales que podrías mejorar para que quede más sólido y profesional. Voy directo a lo importante 👇

🔧 1. Parámetro `ctx` inconsistente

En `drawPixel` pasas `ctx` como parámetro, pero en el resto de funciones usas el `ctx` global.

👉 **Qué corregir:**

- O pasas `ctx` a **todas** las funciones
- O eliminas `ctx` como parámetro en `drawPixel`

✔️ Mantener consistencia evita errores si luego reutilizas funciones.

🎯 2. `randomEntre5y10()` no se está usando

La función está bien, pero no aparece en ninguna parte del código.

👉 **Qué falta:**

- Usarla para generar radios, lados o posiciones aleatorias
- O eliminarla si no hace parte de la tarea

📐 3. `getPolygonVertices` no se integra con Bresenham

Calculas los vértices, pero no estás dibujando el polígono.

👉 **Qué falta:**

- Conectar cada vértice con `bresenhamLine`
- Cerrar la figura (último vértice con el primero)

⚠️ 4. Posible mejora en Bresenham

Tu implementación está bien, pero:

👉 **Qué podrías mejorar:**

- Agregar valor por defecto al `color`
- Validar que los puntos sean enteros (aunque ya lo haces indirectamente)

🔵 5. Punto medio del círculo (detalle técnico)

Tu implementación es funcional, pero hay una pequeña variación común:

👉 **Detalle:**

- Normalmente se usa `+3` y `+5` en vez de `+1` en algunas versiones
- No está mal lo tuyo, pero depende de la formulación que te enseñaron

✔️ Esto no es error, solo una posible diferencia teórica

🧼 6. Limpieza del canvas (falta)

No tienes una función para borrar antes de volver a dibujar.

👉 **Qué falta:**

- Algo como `ctx.clearRect(...)`

🎨 7. Tamaño del píxel fijo

Siempre dibujas píxeles de tamaño 1.

👉 **Mejora opcional:**

- Permitir cambiar el tamaño para visualizar mejor

🧠 8. Organización general

Está bien, pero podrías mejorar:

👉 **Sugerencias:**

- Agrupar funciones por tipo (líneas, polígonos, círculos)
- Separar lógica matemática de renderizado

🚀 Resumen rápido

Lo más importante a corregir:

- ⚠️ Consistencia con `ctx`
- ⚠️ Falta usar `getPolygonVertices` para dibujar
- ⚠️ Función random sin uso
- ➕ Agregar limpieza de canvas

Si quieres, en el siguiente paso te puedo decir ↪ **qué te podrían evaluar mal en un parcial con este código** (modo profe 👀).

```
/**
 * Universidad UMNG - Facultad de Ingeniería Multimedia
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: Jennifer Barahona Suarez
 * * Tarea: Implementar los algoritmos de rasterización
manual.
 */

//Obtenemos el contexto del canvas para dibujar
const canvas = document.getElementById("myCanvas");
```

```
const ctx = canvas.getContext("2d");

//Funcion para crear un número aleatorio entre 5 y 10
function randomEntre5y10() {
    return Math.floor(Math.random() * 6) + 5;
}

// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#a200ff") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1 , 1);
}

/**
 * Algoritmo del punto medio para circunferencias
 * cx, cy: centro de la circunferencia
 * r: radio de la circunferencia
 * color: color de la circunferencia
 * El parámetro p decide cuándo decrementar Y
 */
function drawCircle(cx, cy, r, color = "#13bddb") {
    let x = 0;
    let y = r;
    let p = 1 - r;

    while (x <= y) {
        plotCirclePoints(cx, cy, x, y, color);

        x++;

        if (p < 0) {
            p += 2 * x + 1;
        } else {
            y--;
            p += 2 * (x - y) + 1;
        }
    }
}

function plotCirclePoints(cx, cy, x, y, color) {
```

```
    drawPixel(ctx, cx + x, cy + y, color);
    drawPixel(ctx, cx - x, cy + y, color);
    drawPixel(ctx, cx + x, cy - y, color);
    drawPixel(ctx, cx - x, cy - y, color);

    drawPixel(ctx, cx + y, cy + x, color);
    drawPixel(ctx, cx - y, cy + x, color);
    drawPixel(ctx, cx + y, cy - x, color);
    drawPixel(ctx, cx - y, cy - x, color);
}

/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */

function bresenhamLine(x0, y0, x1, y1, color) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx;
    if (x0 < x1) {
        sx = 1;
    } else {
        sx = -1;
    }

    let sy;
    if (y0 < y1) {
        sy = 1;
    } else {
        sy = -1;
    }

    let error = dx - dy; // parámetro de decisión inicial

    while (true) {
```

```
        drawCircle(ctx, x0, y0, color = "#13bddb")

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * error;

        /**
         * Ajuste del error:
         * Si el error acumulado supera cierto umbral,
         * se corrige desplazando en X o Y para aproximarse
         * a la recta ideal.
         */
        if (e2 > -dy) {
            error -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            error += dx;
            y0 += sy;
        }
    }
}



/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */

function getPolygonVertices(centerX, centerY, sides, radius)
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
    let vertices = [];
```

```
    for (let i = 0; i < randomEntre5y10(); i++) {
        let angulo = (2 * Math.PI * i) / randomEntre5y10();
        let x = centerX + radius * Math.cos(angulo); // Cálculo
de la coordenada X usando coseno
        let y = centerY + radius * Math.sin(angulo); // Cálculo
de la coordenada Y usando seno

        vertices.push({ x: Math.round(x), y: Math.round(y) });
    }

    return vertices;// Retorna un arreglo con las
coordenadas de los vértices
}  asi?
```

Vas bien, pero aquí sí hay **errores importantes** (no solo mejoras). Te marco exactamente lo que debes corregir 👇

❌ 1. Error grave en `bresenhamLine`

JavaScript

👉 Problemas:

- Estás llamando **drawCircle en vez de drawPixel**
- Estás pasando `ctx` donde debería ir `cx`
- Falta el radio (`r`)
- Estás sobrescribiendo `color` dentro de la llamada

👉 **Qué debería pasar conceptualmente:**

- Bresenham dibuja **píxeles**, no círculos

❌ 2. `getPolygonVertices` mal uso del random

JavaScript

```
let angulo = (2 * Math.PI / 1) / randomEntre5y10();
```

👉 Problema:

- Estás llamando `randomEntre5y10()` **dos veces**
- Eso hace que:
  - El número de lados cambie en cada iteración
  - El ángulo sea inconsistente

👉 Resultado:

- El polígono sale deformado o incorrecto

👉 **Qué debes hacer:**

- Guardar el valor random en una variable y usarla siempre

⚠️ 3. Ignoras el parámetro `sides`

Tu función recibe:

JavaScript

Pero luego no usas `sides` para nada.

👉 Eso te lo pueden bajar en nota:

- Porque no estás respetando la firma de la función

⚠️ 4. Inconsistencia con `ctx` (sigue igual)

- `drawPixel(ctx, ...)`
- Pero en otras funciones usas `ctx` global

👉 Ya te lo había mencionado — sigue siendo algo a corregir

⚠️ 5. Orden del código

No es error, pero:

- Normalmente: utilidades → algoritmos → uso
- Aquí mezclas un poco

⚠️ 6. Falta uso real del polígono

Sigues sin:

- Dibujar las líneas entre vértices
- Cerrar la figura

🟡 7. Detalle menor en `drawCircle`

Tu implementación está bien, pero:

- Algunas versiones usan otras constantes (`+3` , `+5`)
- No es error, solo depende del enfoque del profe

🚨 RESUMEN (lo que SÍ debes arreglar sí o sí)

1. ❌ Cambiar `drawCircle` por `drawPixel` en Bresenham
2. ❌ No usar `randomEntre5y10()` múltiples veces dentro del mismo cálculo
3. ❌ Usar correctamente el parámetro `sides`
4. ⚠️ Arreglar consistencia de `ctx`

Si presentas esto así, lo más probable es que te bajen puntos por:
👉 error en Bresenham (grave)
👉 mal uso de parámetros
👉 lógica inconsistente en el polígono

Si quieres, en el siguiente paso te digo **cómo debería comportarse visualmente tu código cuando ya esté bien** (para que sepas si lo lograste).

```
/**
 * Universidad UMNG - Facultad de Ingeniería Multimedia
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: Jennifer Barahona Suarez
 * * Tarea: Implementar los algoritmos de rasterización
manual.
 */

//Obtenemos el contexto del canvas para dibujar
const canvas = document.getElementById("myCanvas");
const ctx = canvas.getContext("2d");

//Funcion para crear un número aleatorio entre 5 y 10
function randomEntre5y10() {
    return Math.floor(Math.random() * 6) + 5;
}

// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#a200ff") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1 , 1);
}

/**
 * Algoritmo del punto medio para circunferencias
 * cx, cy: centro de la circunferencia
 * r: radio de la circunferencia
 * color: color de la circunferencia
 * El parámetro p decide cuándo decrementar Y
 */
function drawCircle(cx, cy, r, color = "#13bddb") {
    let x = 0;
    let y = r;
    let p = 1 - r;

    while (x <= y) {
        plotCirclePoints(cx, cy, x, y, color);
```

```
        x++;

        if (p < 0) {
            p += 2 * x + 1;
        } else {
            y--;
            p += 2 * (x - y) + 1;
        }
    }
}

function plotCirclePoints(cx, cy, x, y, color) {
    drawPixel(ctx, cx + x, cy + y, color);
    drawPixel(ctx, cx - x, cy + y, color);
    drawPixel(ctx, cx + x, cy - y, color);
    drawPixel(ctx, cx - x, cy - y, color);

    drawPixel(ctx, cx + y, cy + x, color);
    drawPixel(ctx, cx - y, cy + x, color);
    drawPixel(ctx, cx + y, cy - x, color);
    drawPixel(ctx, cx - y, cy - x, color);
}

/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */

function bresenhamLine(x0, y0, x1, y1, color) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx;
    if (x0 < x1) {
        sx = 1;
    } else {
```

```
        sx = -1;
    }

    let sy;
    if (y0 < y1) {
        sy = 1;
    } else {
        sy = -1;
    }

    let error = dx - dy; // parámetro de decisión inicial

    while (true) {
        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * error;

        /**
         * Ajuste del error:
         * Si el error acumulado supera cierto umbral,
         * se corrige desplazando en X o Y para aproximarse
         * a la recta ideal.
         */
        if (e2 > -dy) {
            error -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            error += dx;
            y0 += sy;
        }
    }
}



/**
```

```
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */

function getPolygonVertices(centerX, centerY, sides, radius)
{
    let vertices = [];

    for (let i = 0; i < sides; i++) { // ✅ usar sides
        let angulo = (2 * Math.PI * i) / sides; // ✅ usar sides

        let x = centerX + radius * Math.cos(angulo);
        let y = centerY + radius * Math.sin(angulo);

        vertices.push({ x: Math.round(x), y: Math.round(y) });
    }

    return vertices;
} asi esta bien?
```

Ahora sí 👏 — este código ya está **correcto en lo fundamental**. Si lo entregas así, no deberías tener problemas con los algoritmos principales.

Pero te marco los **últimos detalles finos** (nivel "profe exigente") 👇

✅ Lo que ya está bien

- ✔️ Bresenham correctamente implementado
- ✔️ Punto medio del círculo correcto
- ✔️ Uso adecuado de `sides` en el polígono
- ✔️ Cálculo de vértices limpio
- ✔️ Uso de `Math.round` 👍

⚠️ Detalles que aún podrías mejorar

1. 🔁 Inconsistencia con `ctx`

Sigues mezclando estilos:

- `drawPixel(ctx, ...)`
- pero usas `ctx` global en todo

👉 Mejora:

- O eliminas `ctx` como parámetro en `drawPixel`
- O lo pasas a **todas** las funciones

(No es error grave, pero sí de diseño)

2. 🎨 Parámetro `color` en Bresenham

JavaScript

👉 Problema:

- No tiene valor por defecto

👉 Mejora:

- Podrías definir uno como hiciste en `drawCircle`

3. 🧩 Aún no dibujas el polígono

Tu función:

JavaScript

✔️ calcula bien
❌ pero no dibuja nada

👉 Te falta:

- Conectar vértices con `bresenhamLine`
- Cerrar la figura (último → primero)

4. 🧼 Falta limpiar el canvas

Si vuelves a dibujar, todo se acumula

👉 Mejora:

- Función tipo `clearRect`

5. 🎲 `randomEntre5y10()` sigue sin usarse

No es error, pero:

👉 Opciones:

- Usarla para `sides`, `radius`, etc.
- O eliminarla

6. 📦 Organización (detalle menor)

Podrías ordenar así:

1. Utilidades (`drawPixel`, random)
2. Algoritmos (línea, círculo)
3. Geometría (polígonos)

🟢 Conclusión clara

✔️ **Sí, ya está bien**
✔️ Los algoritmos están correctamente implementados
⚠️ Solo te faltan detalles de integración y limpieza

Si quieres subirlo a nivel 💯, el siguiente paso sería:
👉 hacer una función tipo `drawPolygon(...)` que use todo lo que ya hiciste

Ahí es donde normalmente el profe dice: *"ok, sí entendió todo"* 😄

no se como conectar el bresenham ya lo intente pero me da error me podrias decir como es porfa

Te estás trabando en la parte clave: **cómo usar Bresenham para unir los vértices**. No es complicado, pero hay un detalle que si lo haces mal, rompe todo.

Te lo explico claro y directo 👇

🧠 IDEA CLAVE

Ya tienes:

- `getPolygonVertices(...)` → te da puntos `{x, y}`
- `bresenhamLine(x0, y0, x1, y1, color)` → dibuja líneas

👉 Solo necesitas:
conectar cada vértice con el siguiente

🔗 CÓMO SE CONECTAN

Si tienes un arreglo:

JavaScript

Debes hacer:

- v0 → v1
- v1 → v2
- v2 → v3
- ...
- último → primero ⚠️ (esto es lo que más se olvida)

⚠️ ERRORES QUE SEGURO TE ESTÁN PASANDO

❌ 1. No acceder bien a las coordenadas

Mal:

JavaScript

Bien:

JavaScript

❌ 2. Te sales del arreglo

Cuando haces:

JavaScript

en el último índice → da `undefined`

👉 Solución:

- Usar módulo (`%`) para volver al inicio

✅ LÓGICA CORRECTA (IMPORTANTE)

Para cada `i` :

```
punto actual = vertices[i]
siguiente = vertices[(i + 1) % vertices.length]
```

Ese `%` es lo que cierra el polígono automáticamente 🔥

🎯 CÓMO LLAMAR A BRESENHAM

Para cada par de puntos:

```
bresenhamLine(
    x_actual,
    y_actual,
    x_siguiente,
    y_siguiente,
    color
)
```

🧪 EJEMPLO MENTAL

Si tienes un pentágono:

JavaScript

Con el módulo pasa esto:

- 0 → 1
- 1 → 2
- 2 → 3
- 3 → 4
- 4 → 0 ✅ (se cierra)

🚨 SI TE DA ERROR

Revisa esto (casi seguro es uno de estos):

- ❌ `vertices` está vacío
- ❌ `sides < 3`
- ❌ estás pasando mal los parámetros
- ❌ estás llamando la función antes de crear el canvas

💡 TIP PRO

Antes de dibujar líneas, prueba esto:

JavaScript

👉 Si no ves `{x:..., y:...}` → ahí está el problema

🟢 RESUMEN

👉 No necesitas modificar Bresenham
👉 Solo recorrer el arreglo de vértices
👉 Usar `(i + 1) % length` para cerrar
👉 Pasar `.x` y `.y` correctamente

Si quieres, mándame el error exacto que te sale y te digo **exactamente dónde está fallando en tu caso** 👀

pero todo eso se pone fuera de las funciones o se pone en el alguna de las dos?

Buena pregunta — aquí es donde muchos se confunden 👍

👉 **Respuesta corta:**
Eso **NO va dentro de** `bresenhamLine` **ni dentro de** `getPolygonVertices`.

🧠 DÓNDE VA EN REALIDAD

Debes crear una **nueva función**, por ejemplo:

👉 una función tipo: *dibujar polígono*

🧩 POR QUÉ NO VA EN LAS OTRAS

❌ `bresenhamLine`

- Solo dibuja **una línea**
- No sabe nada de polígonos

❌ `getPolygonVertices`

- Solo **calcula puntos**
- No debería dibujar (separa lógica y renderizado)

✅ DÓNDE SÍ VA

👉 En una función aparte que:

1. Llame a `getPolygonVertices`
2. Recorra los vértices
3. Llame a `bresenhamLine`

🔄 FLUJO CORRECTO

```
getPolygonVertices → (obtiene puntos)
↓
función nueva (recorre puntos)
↓
bresenhamLine → (dibuja líneas)
```

⚠️ ERROR COMÚN

Meter el `for` dentro de:

- `getPolygonVertices` ❌
- `bresenhamLine` ❌

👉 Eso rompe la lógica del programa

💡 DÓNDE SE EJECUTA

Esa nueva función la llamas:

- Al final del archivo
- O en un botón / evento

Ejemplo mental:

```
drawPolygon(centroX, centroY, lados, radio)
```

🟢 RESUMEN CLARO

✅ Crear función nueva
✅ No modificar las que ya hiciste
✅ Esa función conecta TODO

Si quieres, te puedo revisar exactamente cómo lo implementes y decirte si está bien o no (como lo haría tu profe 👀).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Figura compuesta por un polígono
regular</title>
</head>
<body>
<form>
    <label>Numero de lados:</label>

    <canvas id="myCanvas" width="500" height="500"
style="border:1px solid #4c0080;"></canvas>
</form>
```

```
</body>
</html>

<script src="script.js"></script> y aqui que me falta?
```

Vas bien, pero a tu HTML le faltan varias cosas clave para que **realmente funcione con tu JavaScript** 👇

⚠️ 1. No tienes un input para los lados

Solo tienes el label:

HTML

👉 Pero el usuario **no puede escribir nada**

✔️ Te falta:

- Un `<input type="number">`

⚠️ 2. No hay botón para ejecutar

Tu código JS no se va a ejecutar solo (a menos que lo hagas automático).

👉 Te falta:

- Un botón tipo “Dibujar”

⚠️ 3. No estás conectando HTML con JS

Necesitas:

- Leer el valor del input
- Llamar a tu función de dibujo

⚠️ 4. El `<script>` está fuera del `<body>`

Funciona a veces, pero no es buena práctica.

👉 Mejor:

- Ponerlo **antes de cerrar** `</body>`

⚠️ 5. Falta evitar recarga del form

Tienes un `<form>`, entonces:

- Al presionar botón → recarga la página ❌

👉 Soluciones:

- Quitar el `<form>`
- O usar `event.preventDefault()`

⚠️ 6. No hay función que se ejecute

Aunque todo esté bien:
👉 Si no llamas a tu función de dibujo → no pasa nada

💡 LO QUE DEBERÍA PASAR

Flujo correcto:

```
Usuario escribe lados
↓
Hace clic en botón
↓
JS toma el valor
↓
Llama a drawPolygon(...)
↓
Se dibuja en canvas
```

🟢 RESUMEN CLARO

Te falta:

- ✔️ Input (`type="number"`)
- ✔️ Botón
- ✔️ Conectar JS con el input
- ✔️ Llamar la función de dibujo
- ✔️ Manejar el form (o quitarlo)

Si quieres, en el siguiente paso ↪ te digo **exactamente qué función debes crear en JS para que todo eso funcione perfecto con tu canvas** 🔥

lo que pasa es que cada vez que se carga la pagina debe aparecer el poligono dibujado

ChatGPT Obtener Plus ×

Pero aún te falta una cosa clave:

⚠️ PROBLEMA PRINCIPAL

Tu HTML carga el script, pero **nunca le dices que dibuje algo**.

👉 Es decir:

- Tienes todas las funciones ✔️
- Pero **no ejecutas ninguna** ❌

✅ SOLUCIÓN CONCEPTUAL

Debes hacer que al cargar la página:

```
se ejecute una función → que dibuje el polígono
```

🔥 OPCIONES (elige una)

✅ Opción 1 (la más simple)

👉 Al final de tu `script.js` :

- Llamas directamente a tu función de dibujo

✅ Opción 2 (más correcta)

👉 Usar un evento de carga:

```
cuando el DOM esté listo → dibujar
```

⚠️ OTRO DETALLE IMPORTANTE

Ya que quieres que sea automático, aprovecha tu función:

👉 `randomEntre5y10()`

Pero **NO dentro de** `getPolygonVertices` (eso ya lo corregiste 👍)

👉 Debes usarla **cuando llamas la función de dibujo**, por ejemplo:

- para el número de lados
- o el radio

🎯 FLUJO FINAL

```
Carga la página
↓
Se ejecuta tu función principal
↓
Generas lados aleatorios
```

```
↓
Obtienes vértices
↓
Conectas con Bresenham
↓
Se dibuja el polígono
```

⚠️ POSIBLES ERRORES SI NO TE FUNCIONA

- ❌ Canvas no existe aún → (script mal ubicado)
- ❌ No llamaste la función
- ❌ Coordenadas fuera del canvas
- ❌ Lados < 3

🟢 RESUMEN

✔️ No necesitas modificar el HTML
✔️ Solo necesitas **ejecutar una función al cargar**
✔️ Usa `randomEntre5y10()` ahí
✔️ Conecta todo (vértices + Bresenham)

Si quieres, mándame cómo hiciste la función de "dibujar polígono" y te digo si ya estás a nivel de entrega o todavía le falta algo 👀

```
<!DOCTYPE html>
<html lang="es">

<head>
    <meta charset="UTF-8">
    <title>Parcial 2 - Computación Gráfica</title>

    <style>
        body {
```

```
            margin: 0;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            background-color: #f0f0f0;
        }

        canvas {
            border: 1px solid black;
        }
    </style>
</head>

<body>

    <canvas id="myCanvas" width="500" height="500">
</canvas>

    <script src="script.js"></script>

</body>

</html> aun no dibuja nada
```

Perfecto, con ese HTML ya está bien armado 👍
👉 Si **no dibuja nada**, el problema ya está 100% en tu `script.js`.

Voy directo a lo que te falta (sin rodeos) 👇

🚨 PROBLEMA REAL

👉 Nunca estás llamando ninguna función de dibujo

Tienes:

- `drawCircle` ✅
- `bresenhamLine` ✅

- `getPolygonVertices` ✔

Pero:

❌ **no hay ninguna ejecución**

✅ LO QUE TE FALTA (CLAVE)

Debes hacer algo como esto (conceptualmente):

```
1. Generar lados (random o fijo)
2. Obtener vértices
3. Recorrer vértices
4. Llamar a bresenhamLine
```

🔥 CHECKLIST RÁPIDO

Revisa si en tu `script.js` tienes algo así:

1. ¿Hay una llamada al final del archivo?

👉 Si no hay algo que se ejecute → NO se dibuja nada

2. ¿Estás creando los vértices?

👉 Algo tipo:

```
vertices = getPolygonVertices(...)
```

3. ¿Estás recorriendo los vértices?

👉 Necesitas un `for`

4. ¿Estás llamando Bresenham?

👉 Dentro del `for`

5. ¿Estás cerrando el polígono?

👉 Con:

```
(i + 1) % vertices.length
```

⚠️ ERRORES MÁS COMUNES (seguro es uno de estos)

- ❌ Nunca llamaste la función
- ❌ `vertices` está vacío
- ❌ No hiciste el `for`
- ❌ No usaste `.x` y `.y`
- ❌ Coordenadas fuera del canvas

🧪 PRUEBA RÁPIDA (haz esto ya)

Agrega esto al final de tu `script.js`:

JavaScript

👉 Si no aparece en consola:
➡️ Tu script no está cargando

👉 Si aparece:
➡️ El problema es que no estás dibujando nada

🎯 PISTA FINAL (la más importante)

Tu código actual es como tener:
🧠 el cerebro (funciones)
❌ pero no presionas el botón de "ejecutar"

🟢 RESUMEN

✅ Tu HTML está bien
❌ Falta ejecutar el dibujo en JS
👉 Necesitas una función que conecte TODO
👉 Y llamarla automáticamente

Si quieres, mándame tu `script.js` completo y te digo exactamente en qué línea está el fallo 👀

puedes darme el bloque de codigo que me falta porfa

Sí — te falta **ese bloque que conecta todo y se ejecuta al cargar**.
Aquí lo tienes 👇 (esto va **al final de tu** `script.js`):

```javascript
// Función que dibuja el polígono usando todo lo que ya hiciste
function drawPolygon() {
    const sides = randomEntre5y10(); // número de lados aleatorio (5 a 10)
    const radius = 100;              // puedes ajustar el tamaño
    const centerX = canvas.width / 2;
    const centerY = canvas.height / 2;

    const vertices = getPolygonVertices(centerX, centerY, sides, radius);

    for (let i = 0; i < vertices.length; i++) {
        let current = vertices[i];
        let next = vertices[(i + 1) % vertices.length]; // cierra el polígono

        bresenhamLine(current.x, current.y, next.x, next.y, "#000000");
    }
}
```

JavaScript

🔥 Qué hace este bloque

- Usa tu `randomEntre5y10()` ✅
- Llama `getPolygonVertices()` ✅
- Conecta con `bresenhamLine()` ✅
- Cierra el polígono ✅
- Se ejecuta automáticamente ✅

Si con esto no se dibuja, ya sería algo muy puntual (tipo canvas o coordenadas), pero con lo que llevas debería funcionar sin problema 🚀

```
function drawPolygon(centerX, centerY, sides, radius, color
= "#000000") {
    let vertices = getPolygonVertices(centerX, centerY, sides,
radius);

    for (let i = 0; i < vertices.length; i++) {
        let next = (i + 1) % vertices.length; // conecta el
último con el primero

        bresenhamLine(
            vertices[i].x,
            vertices[i].y,
            vertices[next].x,
            vertices[next].y,
            color
        );
    }
}  pero yo tengo esta solo añade lo que le falta
```

Tu función está **bien hecha** 👍 — no le falta lógica interna.

👉 Lo que te falta NO es modificarla, sino **usarla (llamarla)**.

✅ SOLO AÑADE ESTO (al final de tu `script.js`)

```javascript
// Parámetros para dibujar
const centerX = canvas.width / 2;
const centerY = canvas.height / 2;
const sides = randomEntre5y10();
const radius = 100;
```

JavaScript

🔥 Qué hace esto

- Define el centro del canvas ✔
- Usa tu random ✔
- Llama tu función ✔
- Hace que **se dibuje al cargar** ✔

⚠️ Si aún no se ve nada

Revisa rápido:

- ¿El `radius` no es muy grande? (ej: 200+ puede salirse)
- ¿El canvas sí mide 500x500?
- ¿No hay errores en consola?

🟢 Resumen

👉 Tu función ya está correcta
👉 Solo faltaba **invocarla**

Si quieres, luego te enseño cómo hacer que se dibujen **varios polígonos aleatorios automáticamente** (queda muy pro 👀).

para trazar un circulo en cada vertice tengo que usar la funcion drawcircle o tengo que hacer otra cosa?

Vas por buen camino 👌

👉 **Sí debes usar** `drawCircle`, no necesitas otro algoritmo.

Pero hay un detalle importante:

🧠 IDEA CLAVE

Ya tienes los vértices:

JavaScript

👉 Cada vértice tiene:

JavaScript

Entonces:

👉 Solo debes **llamar** `drawCircle` **en cada vértice**

🔥 DÓNDE HACERLO

👉 Dentro del mismo `for` donde dibujas el polígono

Porque ahí ya estás recorriendo todos los puntos ✔️

🎯 LÓGICA

Por cada vértice:

```
dibujar línea
dibujar círculo en ese punto
```

⚠️ ERRORES COMUNES

❌ 1. Intentar modificar `drawCircle`

No necesitas tocar esa función.

❌ 2. Dibujar fuera del `for`

Si lo haces fuera:

- Solo dibujas 1 círculo ❌

❌ 3. Radio muy grande

Si el círculo es muy grande:

- Se ve raro o tapa el polígono

👉 Usa un radio pequeño (ej: 3–6)

❌ 4. Pasar mal los parámetros

Recuerda:

```
drawCircle(cx, cy, r, color)
```

👉 `cx` y `cy` son el vértice

🟢 RESUMEN

✔️ Usa tu `drawCircle`

✔️ Llámalo dentro del `for`

✔ Usa `vertices[i].x` y `vertices[i].y`

✔ Radio pequeño

Si quieres subirle nivel 🔥, puedes hacer:

👉 ↪ un color distinto para cada vértice

👉 o ↪ que el radio sea random

y te queda visualmente mucho más pro 😄

```
function drawPolygon(centerX, centerY, sides, radius, color
= "#e210f5") {
    let vertices = getPolygonVertices(centerX, centerY, sides,
radius);

    for (let i = 0; i < vertices.length; i++) {
        let next = (i + 1) % vertices.length; // conecta el
último con el primero

        bresenhamLine(
            vertices[i].x,
            vertices[i].y,
            vertices[next].x,
            vertices[next].y,
            color
        );
    }

    // Dibujar circunferencia en cada vértice (R/4)
    for (let i = 0; i < vertices.length; i++) {
        drawCircle(vertices[i].x, vertices[i].y, radius / 4);
    }
}
```
asi esta bien o falta algo?

Está **casi perfecto** 👏 pero hay un detalle importante que puede jugarte en contra dependiendo de lo que te pidan exactamente.

✅ Lo que está bien

- ✔️ Conectas vértices con Bresenham
- ✔️ Cierras el polígono correctamente
- ✔️ Recorres los vértices para dibujar círculos
- ✔️ Usas `drawCircle` correctamente

⚠️ DETALLE IMPORTANTE

🎯 1. Tamaño del círculo (`radius / 4`)

👉 Puede ser **demasiado grande**

- Si el polígono es pequeño → los círculos se van a **encimar**
- Visualmente puede verse mal o confuso

👉 Mejor práctica:

- Usar un valor fijo pequeño (ej: 5–10)
- O algo proporcional pero más pequeño (ej: `radius / 10`)

⚠️ 2. Color de los círculos

Ahora mismo:

JavaScript

👉 Estás usando el color por defecto (`#13bddb`)

✔️ No es error, pero:

- Podrías querer mismo color del polígono
- O diferenciarlo intencionalmente

💡 3. Optimización (no obligatoria)

Estás haciendo **dos** `for` **separados**:

👉 Podrías hacerlo en uno solo (más eficiente)

Pero:

✔️ Lo que tienes está **correcto**
❌ No te van a bajar puntos por eso

🧠 4. Validación (nivel pro)

Podrías asegurar:

JavaScript

🟢 CONCLUSIÓN

👉 **Sí, está bien y funcional**
👉 Ya cumple con lo que normalmente piden en un parcial

🔥 Si lo quieres dejar nivel 100

Solo ajusta:

- tamaño del círculo (más pequeño)
- opcional: color

Si lo entregas así, ya estás en nivel de:
👉 *"sí entendió Bresenham + punto medio + integración"* 😄

Listo gracias :D

¡De una! 😄 Me alegra que lo hayas sacado.

Si luego quieres subirle nivel (animarlo, hacerlo interactivo o meterle más algoritmos), me dices y lo armamos 💻✨